

Neil Ghani's *Containers for Typed Agentic AI* describes an algebra of interfaces. An interface is a pair. The first half is the prompts you may send. The second half gives, for each prompt, the type of response that would answer it.

Such interfaces compose, in four different ways. Three of them appear in this game.

⊗	tensor	prompts both sides, and requires a response from each
◁	sequential	prompts one side, and computes the second prompt from the first response
×	product	offers both sides, and takes a response from exactly one
+	sum	takes a prompt belonging to one side or the other, and answers with that side's own response

The sum is routing. We only use the first three in this game.

A morphism of containers $(P, R) \rightarrow (Q, T)$ is a pair of functions.

$$u : P \rightarrow Q \qquad f : (p : P) \rightarrow T (u p) \rightarrow R p.$$

The mapping u delegates a task. The mapping f amalgamates a response. So a container morphism has two flows, prompts moving forwards and responses moving backwards.

Control flows *down* through u . Data flows *back up* through f .

CLI tools: The forward map u on shapes is implemented as a Unix CLI tool call.

This works because a command-line tool is already a container. It receives one prompt, its argument vector. It produces a response whose *type depends on which prompt it was*.

TypeScript async event handlers: The backwards map f on positions is implemented with TypeScript async event handlers.

Typed agents

Note that the Claude Code harness is itself a CLI tool, and thus a container. We can simply “plug in” the `claude` binary as our CLI tool, and all the machinery developed in this note works exactly the same.

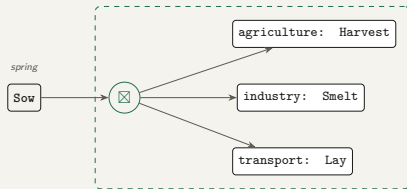
A note on formal verification

The architecture of the game is motivated by Ghani's theory of container combinators, but the design is not formally verifiable. This was never the point. It is a “fake”, but it is a fake that works.

Stalin: The Game

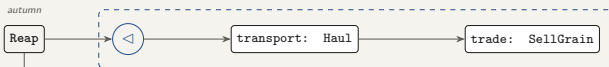
This is a game about Stalin and his First Five-Year Plan. You are given the Soviet Union in 1928 and five years to industrialise it. You must also win the war in 1941.

This was a tragic episode of history and I don't wish to make light of the story. It is for illustrative purposes only. It is also a history lesson.



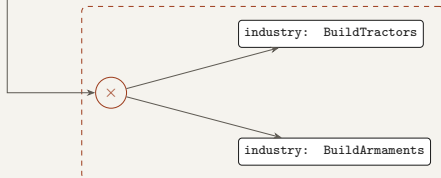
the tensor

three sectors, one year. All three must answer



sequential composition

what you may sell is decided by what the railway moved



the product

both are asked. Exactly one answers, because there is one stock of steel

Everything is a file

An actual file on a disk, a terminal, a network connection, a source of random bytes. All of them are presented by the kernel to user space through the same interface. So the same two calls reach all of them.

That interface to the kernel

```
read (n, buffer, count)  
write(n, buffer, count)
```

The argument `n` is the file descriptor.

The shell

The shell is not part of the kernel and has no privilege you do not have. It is an ordinary process running in user space with your identity, and its purpose is to start other processes. That is all it is. A process whose job is starting other processes.

Pipes

A pipe is a buffer the kernel will hand out on request. It comes with two descriptors. One to write into, one to read out of. The symbol `|` is the same redirection as `>`, with a pipe where the file was. The shell asks for a pipe, forks twice, puts the writing end in the left program's slot 1 and the reading end in the right program's slot 0, and execs both.

The Unix philosophy

Write programs that do one thing and do it well.

Write programs to work together.

Write programs to handle text streams, because that is a universal interface.

A CLI tool is already a container.

A command-line tool receives one prompt, its argument vector. It produces a response whose *type depends on which prompt it was*.

For example the command `git log` answers with a list of commits. The command `git status` answers with a working tree. Those are not the same kind of thing, and which kind you get is fixed by the word after `git`.

The interface is documented in the man pages. But there is *no type checker*. Nothing verifies whether the output from one CLI tool piped into the input of another has the right type.

Deliberately simple

The designer of the HTTP protocol, Roy Fielding, made the same kind of design decision that Unix made. And it paid off. Just as every server in the world today runs Unix, every web app in the world today communicates via HTTP.

There are four types of HTTP request. We only use two of them.

Method	Means
GET	read something, change nothing
POST	create something

A simple web app runs a database on the server. When the server receives a GET request it reads the database and returns the result to the client, or returns an error. When the server receives a POST request it either writes to the database or returns an error.

The command `curl` is just another Unix CLI tool. It sends one HTTP request across the wire and prints the response to the standard output.

Note that since “everything is a file” the Unix kernel handles all the networking, and `curl` just thinks it is reading to and writing from a file.

We say that the server is “listening” on port 8099. What this really means is that the kernel has created a socket with unique ID number 8099 and handed it as a unique file descriptor to the server. The kernel handles all the networking and the server thinks it is reading and writing to a file. So we can forget about “the wire”.

This is very similar to a Unix pipe.

JSON is short for JavaScript Object Notation. It is a recursive text format for data. Just like HTTP it is deliberately simple. It has four types of “atomic” data: numbers, strings, booleans and null. Then it has two recursive constructors: array and object.

```
value ::= string | number | boolean | null  
       | '[' value* ']'  
       | '{' (string ':' value)* '}'
```

Unchecked types

A CLI interface is documented in the man pages, and nowhere enforced.

The same is true one level up. The interface between the CLI tool making an HTTP request and the TypeScript handler that answers it is fully specified in the design spec, and nowhere enforced.

It must be maintained by the developer.

The design used by deno keeps one flow of control and never lets it block. Every operation that would have waited is written in three parts instead.

1. Start it. Ask for the thing, and do not wait for it.
2. Give the flow of control back. Stop, so that the other conversations can be served.
3. Arrange to be resumed. Leave behind what should happen when the response arrives.

Something has to hold those arrangements. That something is the event loop.

Async hell

Functions which depend on other functions which in turn depend on other functions form a tree. Depending on how long each function takes to complete, they may return in any order which is a linear extension of that tree.

For a complex dependency tree, writing out all the `async` and `await` keywords to record the nesting structure is a nightmare. It is commonly referred to as `async hell`. This is exactly the problem that Ghani's container formalism solves.

The tree is a dependency order, which is a partial order. The event loop produces some topological sort of it, and which one you get is decided by whichever promise settles first.

That is the gap the four combinators close. The tensor $\boxtimes$ says that two subtrees are independent and may interleave. The sequential $\triangleleft$ says that one must precede the other. The combinators are a language for writing down the partial order, so you no longer have to hand-schedule a linear one.

The three fields of a Book form a finite union:

```
type Field = "id" | "title" | "author";
```

Notice that the three fields do not hold the same kind of thing. An id is a number. A title and an author are strings. That fact can be written down as a function, on the level of types.

```
type FieldType<F extends Field> =  
  F extends "id" ? number : string;
```

The function below returns a book field. Its result type is calculated from which field you ask for. It looks a little bit like a dependent type, but that is not quite what it is.

```
function field<F extends Field>(b: Book, f: F): FieldType<F> {  
  return b[f] as FieldType<F>;  
}
```

```
const n = field(book, "id");      // n : number  
const s = field(book, "title");   // s : string  
const t = field(book, "author");  // t : string
```


The fake container

An interface is a pair. The prompts you may send, and for each prompt the type of response it can receive.

Here `Field` is the type of the prompts, and `FieldType` is the type of the response for each prompt.

So `(Field, FieldType)` is a container, and `field` is an inhabitant of it.

Everything is a CLI tool

The TypeScript web server deno is itself a CLI tool, and it can execute other CLI tools. Claude himself is a large language model running on Anthropic's servers. The Claude Code harness is something else entirely. It is a TypeScript program running on Unix as a CLI tool. So deno can delegate to Claude Code.

Formal verifiability

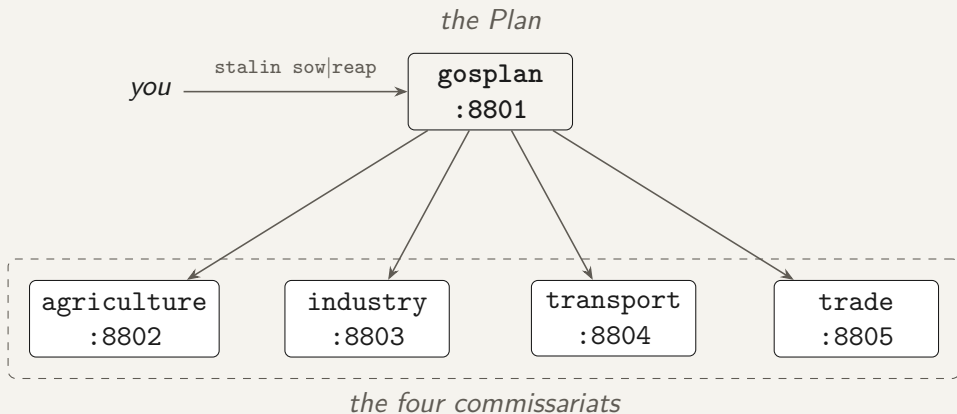
The short answer is that this design cannot be formally verified. The algebra motivates the architecture. It does not verify it.

The algebra is a design vocabulary, and it was used as one.

The algebra is the reason the game is five processes rather than one. A morphism has a level above and a level below, and making the boundary a socket makes that claim falsifiable.

The algebra is the reason each turn is a named composite rather than a block of code. The algebra offers four ways to combine, and forces the designer to say which.

The game runs as five programs, each listening on a port.
The player talks only to the Plan. The Plan delegates to the
four commissariats beneath it.



Conclusion

What has been built is a *fake*. Containers for event-driven programming, faked with CLI tools and TypeScript `async` event handlers.

The nicest thing about this fake is that you can substitute `claude` for any of the CLI tools.

The impersonation has three parts.

- ▶ A container (P, R) becomes a tagged union together with a lookup indexed by its tags. That works, and works only, because the tag set is *finite*. Over a finite index a dependent type is just a table, and a table is something TypeScript can write down.
- ▶ The inhabitant $(p : P) \rightarrow R\ p$ becomes the handler record. The compiler enforces its totality, because it is a mapped type over that same union.
- ▶ A morphism becomes an `async` function. That is where the two directions meet.

```
const reply = await lower.run(u(prompt));    // the prompt, going down
return f(prompt, reply);                     // the reply, coming back
```

Even though this implementation of Ghani's container combinators is not formally verifiable in the way that Kodamai would perhaps wish, Claude can still build lots of cool stuff with it.

Very quickly.